

Perception Sensors and ROS Integration

Importance of perception sensors in robotics.

Sensors are devices that are capable of perceiving and converting environmental information into electrical signals. Sensors are divided into two main categories based on their operating principles. Proprioceptive sensors are used to measure the internal values of a dynamic system (such as a robot), like motor speed, robot arm joint angles, and robot pose. Exteroceptive sensors, in contrast, acquire information from a robot's environment, such as, measurements of distance, light intensity, sound amplitude, temperature, force magnitude, gas concentration.

Why ROS is used for integrating these sensors

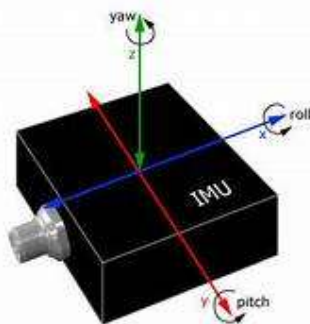
ROS drivers are special nodes that publish and subscribe to topics related to the sensor data and commands. Drivers already developed and tested by ROS community are used, saving time and effort. Sensors can be combined which are of the same type if they are compatible or can switch between sensors. Suppose switching from ultrasonic to infrared sensors to measure distance or using two ultrasonic sensors for accuracy.

Publisher-subscriber communication framework is followed which is a modular approach and allows easy sensor replacement without changing the entire system. Standard messages types are there for each sensors simplifying processing. Eg: **IMU Data:** sensor_msgs/Imu, **Stereo Cameras:** sensor_msgs/Image. Also before deploying the sensors on real robot, ROS allows testing in gazebo and rviz.

SENSORS

IMU(Inertial Measurement Unit)

Working principle: Calculate and reports an exact force of body, angular rate as well as the direction of the body, which can be achieved by using a blend of 3 sensors like Gyroscope(for rotational rate), Magnetometer(heading reference), and Accelerometer(for linear acceleration) for each axis used for each of the 3-vehicle axes like roll, yaw, and pitch.



These sensors include a 3-axis accelerometer and gyroscope. So, this would be measured as a 6-axis IMU sensor. These can also comprise an extra 3-axis magnetometer, so it can be considered like a 9-axis IMU.

A 6-axis IMU consists of : 3 axis accelerometer and a 3 axis gyroscope.

A 9 axis IMU consists of : 3 axis accelerometer ,a 3 axis gyroscope and a 3 axis magnetometer.

The magnetometer helps determine the absolute orientation of the device using Earth's magnetic field. A 6-axis IMU tracks motion but cannot determine absolute orientation over time because gyroscopes drift. A 9-axis IMU corrects this drift using the magnetometer, allowing more accurate direction sensing.

Some IMU sensors:

MPU-6050- 6 axis, Arduino projects and small robots

MPU-9250/MPU-9255 -9 axis , VR headset and mobile

BNO055(Bosch) – 9 axis, built in sensor fusion, outputs orientation directly

LSM9DS1(STMicroelectronics) -9 axis

ICM-20948 -9 axis, improved version of MPU-9250 , higher accuracy and lower power

Xsens MTi-300 – 9 axis . advanced fusion

VN-100/VN-200(VectorNav) – 9 axis , GNSS integration

Sensor fusion: is process of combining data from multiple sensors to get accurate results.It combines using math algorithms like – Complementary filter, Kalman filter, Madgwick filter.

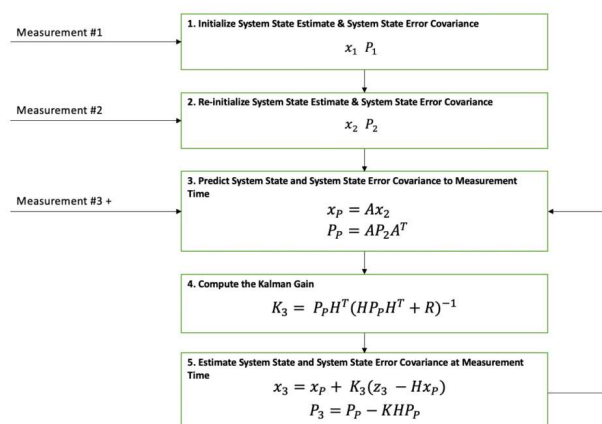
Math algorithms to combine data from sensors:

Complementary filter: This sensor can be used when sensors provide complementary information that is one is good at short term while other at long-term OR low vs high frequency. It work in 1D and 2D so it struggles with 3 sensor fusion (gyro+ accel+ magneto) we will have to create 3 separate filters.

- Gyro + accel for pitch
 - Gyro + accel for roll
 - Gyro + magneto for yaw
- More error prone and complicated but prevents gimbal lock.

Working: $\text{angle} = \alpha * (\text{angle} + \text{gyro_rate} * dt) + (1 - \alpha) * \text{accel_angle}$

- $\text{gyro_rate} * dt$ = change in angle from gyro
- accel_angle = angle from accelerometer
- α = weighting factor (e.g., 0.98)- $\alpha = 0.98 \rightarrow 98\%$ trust in gyro, 2% in accelerometer.



Kalman filter: is a set of mathematical equations that provides an efficient computational (recursive) solution of the least-squares method. The Kalman filter addresses the general problem of trying to estimate the state of a discrete-time controlled process (the **state** is a set of **variables** that describe the **current condition of a system** at any point in time) example: Kalman filter to estimate tilt angle of robot- angle(current tilt) and angular velocity (how fast the angle is changing) State $X = [\theta, \omega]^T$

It estimates the next angle based on current angle. Takes actual sensor readings and compare with the predicted values. Calculate Kalman Gain- how much we can trust the new measurement. Correct the predicted angle using the new measurement.

Madgwick filter: combines data from:

- Gyroscope (angular velocity)- Uses the **Coriolis effect**: A vibrating structure inside the chip changes motion when the device rotates. change is sensed and turned into an electrical signal representing rotation speed.
- Accelerometer (linear acceleration $\rightarrow$ orientation)- Uses a tiny **mass on a spring** inside a MEMS (Micro-Electro-Mechanical System) chip, When the device moves, the mass shifts, and this shift is converted into **electrical signals** representing acceleration
- Magnetometer (magnetic field $\rightarrow$ yaw)- Tiny sensors detect the magnetic field strength on each axis.

It gives you accurate orientation (pitch, roll, yaw) in quaternion or Euler angles. It handles sensor noise better than complementary filter, is lighter than Kalman filter and provides real-time performance. Complementary filter/ kalman filter uses euler angles while madgwich uses quaternions.

Working: It predicts orientation using gyroscope data and update the orientation using quaternion math- $\dot{q} = 1/2 q \otimes \omega$. Then it compares the predicted (from quaternion) with the measured (from accelerometer) also compare predicted magnetic field with measured magnetic field (from magnetometer). Calculate the error between the prediction and the measured data. Uses gradient descent to adjust the quaternion and reduce the error, update the quaternion.

ROS INTEGRATION

ROS packages:

sensor_msgs/Imu

sensor_msgs/MagneticField

imu_filter_madgwick- Applies the Madgwick filter, Subscribes to /imu/data_raw, publishes to /imu/data.

robot_localization- fuses IMU with GPS using Kalman filters.

Imu_tools- Contains tools like imu_complementary_filter for simple **complementary filtering** of IMU data.

rviz_imu_plugin- RViz plugin for visualizing IMU orientation

gazebo_ros_imu_plugin- Simulates an IMU in **Gazebo** and publishes to ROS topics

Message types :

sensor_msgs/Imu :

geometry_msgs/Quaternion orientation → orientation as a quaternion.

geometry_msgs/Vector3 angular_velocity → gyroscope data

geometry_msgs/Vector3 linear_acceleration → accelerometer data

sensor_msgs/MagneticField :

geometry_msgs/Vector3 magnetic_field- Used for magnetometer data

IMU communicates via I2C, SPI and UART protocols.

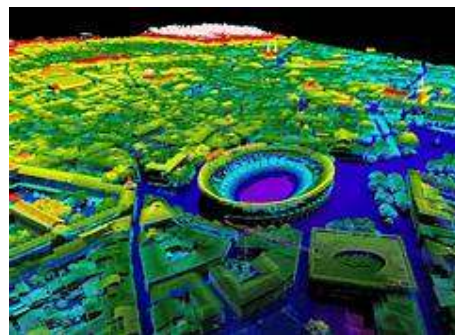
Many IMU's have ROS driver that reads sensor data and publishes to ROS topic , we can install the package and run the code or create our own ROS node . Drivers are software nodes (usually part of pre-made packages). They read raw data from sensors , wrap it into ROS messages and publish on ROS topics. A ROS driver is a ROS node that specifically interfaces with hardware. talks to sensors or actuators via USB, I2C, SPI, serial, etc. Then it converts real hardware data into ROS messages for the rest of the system to use. Interfacing with hardware means the code is directly reading from or controlling a real physical device using wires/signals and communication protocols.

Driver node gets raw data directly from the hardware (the IMU), so it doesn't need input from any other node. Instead, it acts as the source of data for the rest of the ROS system.

To make a simple node into a driver node: You write a Python or C++ node that:

1. Opens the serial port
2. Reads raw IMU data
3. Wraps it in a ROS message (sensor_msgs/Imu)
4. Publishes to /imu/data_raw

LiDAR(Light detection and ranging)



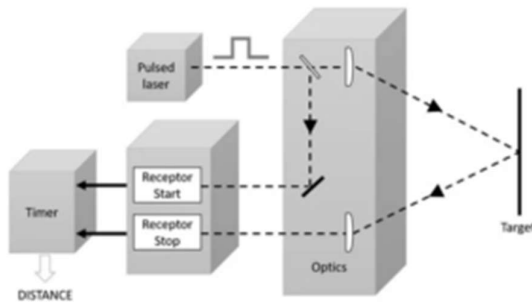
LiDAR stands for light detection and ranging. Technology consists in sending a laser beam to the target and measuring the reflected light with a photodetector to determine the distance to the target and this way generate a precise map of the surrounding environment. Velodyne HDL-64E

LiDAR works based on two sets of measurements:

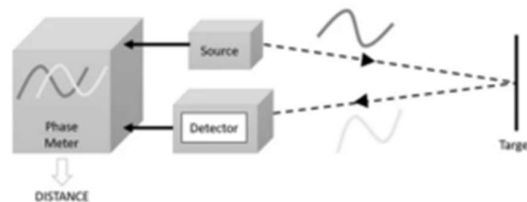
- Position and pointing directions

- Distance –

- 1) employing pulse laser and time of flight that is the time it takes the light to reach the surface and return to the source $D = c \cdot \Delta T / 2$



- 2) using AMCW that is amplitude modulated continuous waveform laser, measuring the phase. The phase shift between the incident and returning light is employed to deduce the distance: $D = c/2 \cdot \Delta\Phi / (2 \cdot \pi \cdot f_M)$ It can only measure til 100m without ambiguity.



There are two main types of LiDAR: airborne and terrestrial. Airborne LiDARs are usually installed in an aircraft or a helicopter pointing downwards towards the ground and normally they measure an angular range of 180°. On the other hand, terrestrial LiDARs can be 1-D or 2-D, covering 180° or 360° in this last case.

- LiDAR emits laser pulses — not just one, but thousands per second
- LiDAR **rotates or spins** to cover all angles
- From the rotation angle, elevation angle, and distance (measured by Time of Flight), the LiDAR calculates the exact 3D position of the point relative to the sensor.(X,Y,Z)
- Result =Massive collection of 3D points - A Point Cloud is just a huge list of 3D points collected in real time. These points together form the shape of the environment — like a 3D scan.

APPLICATION OF LIDAR IN SLAM:

Lidar SLAM, or Light Detection and Ranging Simultaneous Localization and Mapping, is a process that enables robots or autonomous systems to build a map of an unknown environment while simultaneously determining their position within that map. Localization refers to determining the robot's position and orientation within the map, while mapping involves creating a spatial representation of the environment.

Key components of a SLAM system:

- Every SLAM system has some sort of instrument or gadget that enables a robot or other vehicle to see and measure its surroundings. Sonar, LiDAR laser scanner technology, cameras, and other image sensors can all be used for this.
- A SLAM system must have some form of software that aids extract the sensory data gathered by the range measurement device and use it to recognise landmarks in an unknown environment.



SLAM algorithms typically consist of two main components:

- 1) prediction step - also known as motion or odometry update, estimates the robot's new position based on its previous position.
- 2) correction step- also known as observation or sensor update, refines the predicted position using measurements from the environment

LiDAR sensors provide accurate and high resolution data about the environment. SLAM algorithms use the Lidar data to identify and track landmarks in the environment. By comparing the observed landmarks to the landmarks stored in the map, the algorithm can determine the robot's position and orientation relative to the map. As the robot explores the environment and collects new Lidar data, the map is continuously updated, and the robot's pose is refined. This iterative process enables Lidar SLAM systems to create accurate maps and provide real-time localization in various applications

Examples :

1. LOAM (LiDAR Odometry and Mapping)

LiDAR Handling:

- Designed **specifically for 3D LiDAR** (e.g., Velodyne).
- Separates **odometry and mapping** into two threads.

Key Feature:

- **High accuracy 6-DoF pose estimation** using edge and planar features extracted from point clouds.

2. Cartographer (by Google)

LiDAR Handling:

- Supports both **2D and 3D LiDAR**.
- Can **fuse LiDAR with IMU and other sensors** (optional but beneficial).

Key Feature:

- Uses **submap-based scan matching** and **pose graph optimization**.

3. Hector SLAM

LiDAR Handling:

- Designed for **2D LiDAR** only.
- Does **not require odometry or IMU** — relies entirely on **scan matching**.

Key Feature:

- **High-speed scan matching** using Gauss-Newton optimization.

4. GMapping

LiDAR Handling:

- Uses **2D LiDAR**.
- Requires **odometry input**; LiDAR refines pose estimates.

Key Feature:

- **Particle filter-based SLAM**.

Popular Applications of Lidar SLAM in Real Life: Self driving cars, cleaning robots (object detection and avoidance), SLAM drones used for surveying and mapping forms map of the surrounding.

ROS INTEGRATION:

What is processing pipelines ?

A processing pipeline refers to the step-by-step flow of data from the sensor (LiDAR) through various software nodes in ROS. data moves through stages — from raw data → ROS driver → data formatting → processing (e.g., SLAM, obstacle detection) → use in robot actions.

ROS driver nodes(rplidar_ros -2D and velodyne_pointcloud- 3D)

Rplidar_ros – ROS driver for RPLIDAR sensors and publishes data as sensor_msgs/LaserScan- contain range reading(RPLIDAR stands for RoboPeak LiDAR)

Velodyne_pointcloud - ROS driver for Velodyne LiDARs publishes data as sensor_msgs/PointCloud2 – represent a 3D point cloud with XYZ coordinates.

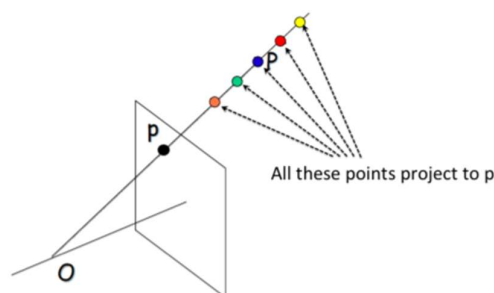
STEREO CAMERAS



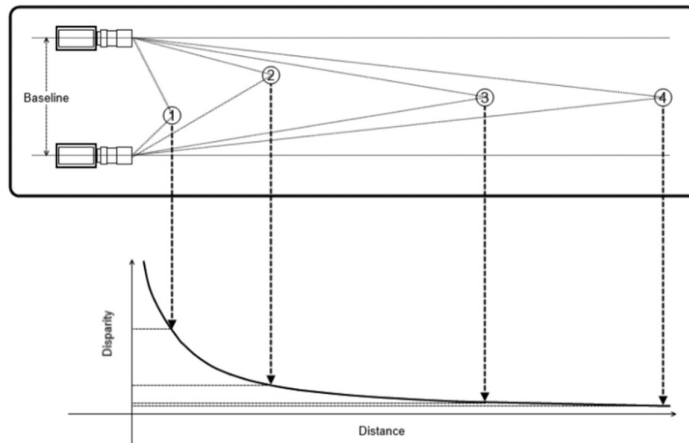
A stereo camera is a type of camera with two or more lenses with a separate image-sensor or film frame for each lens. Simulate human binocular vision, and therefore gives it the ability to capture three-dimensional images.

3D MAPPING: Stereo cameras can create 3D maps of an environment by capturing depth information and combining it over time as the camera moves. After obtaining the depth map, we convert it into a 3D point cloud. This involves mapping each 2D image pixel and its corresponding depth value into 3D space coordinates (X, Y, Z). To build a complete 3D map, we need to merge these point clouds.

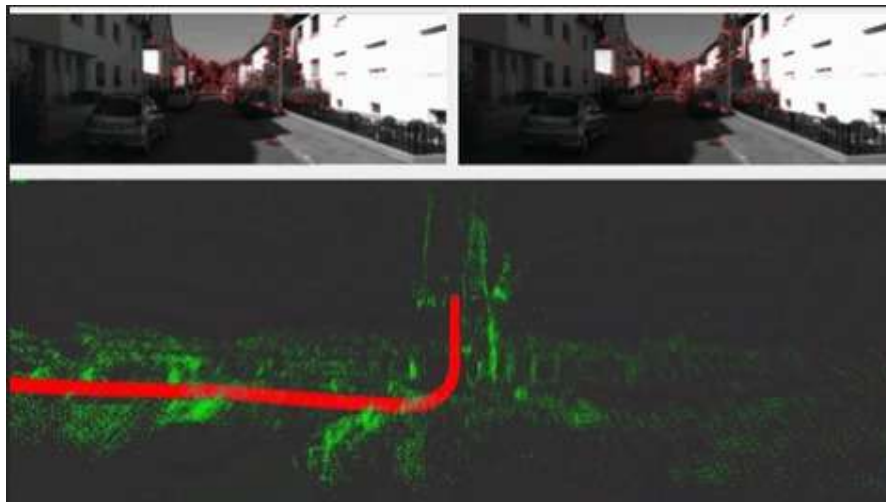
Problem it solves : It is not possible to estimate the distance (depth) of a point object 'P' from the camera using a single camera 'O'. This is because however close or far 'P' is on the projective line, it will map to the same point 'p' in the image. To determine the depth of the point, stereo camera is required .



Disparity is the **difference in the position** of a specific point (pixel) in the **left and right images** captured by a stereo camera setup. In a grayscale image, intensity of each pixel shows the amount of disparity. Brighter areas = higher disparity = closer objects. Here, the more the disparity is, the closer the object is to the baseline of the camera. If we have a small baseline distance between the two cameras, then the difference/disparity between the two images is going to be small.



Visual odometry is the process of estimating a moving camera's position by processing the individual images taken from it



Stereo camera can estimate the depth of a pixel on an image using a technique called triangulation. This is similar to how humans can sense how far away an object is using both eyes.

How it works: Features are extracted from the left image using FAST (Features from Accelerated Segment Test) feature detector in OpenCV. Features in the left image are matched to features in the right image.

In the ROS (Robot Operating System) environment, there are specialized packages to handle each of these steps.

the stereo_image_proc- package processes stereo images to generate disparity and depth images.

depth_image_proc- converts depth data into point clouds.

Zed_ros – to use ZED camera , official wrapper

These point clouds are then visualized using tools like RViz. To create a consistent and accurate 3D map over time.

ROS Message Types in Stereo Vision

- **sensor_msgs/Image** : to transmit images captured by the camera. both **left and right camera images** are published using this type. It also carries **depth images**, where each pixel's value represents the **distance** from the camera.
- **sensor_msgs/PointCloud2**: represents a 3D point cloud.

In drones and Autonomous Guided Vehicles (AGVs), stereo cameras provide real-time depth information. The left and right images (via sensor_msgs/Image) are processed to compute depth, which is then converted into point clouds (sensor_msgs/PointCloud2).

GPS (Global Positioning System)

GPS sensor consists of surface mount chip which processes signals from GPS satellites using a small rectangular antenna, often mounted on the top of the GPS chip. GPS works based on a process called satellite trilateration.

Satellite Signals: GPS satellites, orbiting about 20,000 km above the Earth, constantly transmit radio signals containing the satellite's position

Receiving Signals: The GPS sensor (receiver) in a robot or device picks up signals from multiple satellites. To determine a precise position, signals from at least four satellites are needed. 3 satellites are used for determining position. 1 satellite is used to correct time error.

Time of Flight Calculation: By calculating how long each signal took to reach the receiver (based on the difference between the time sent and received), the GPS can estimate its distance to each satellite.

Trilateration: Using these distances and the known positions of the satellites, the GPS sensor applies trilateration to compute its 3D position. Trilateration is a mathematical method used to determine the position of an object by measuring its distances from multiple known points.

ROS INTEGRATION

In ROS (Robot Operating System), GPS data fusion is a process that combines GPS readings with other sensor data (like IMU and odometry) to generate a more accurate estimate of a robot's position and movement.

Workflow:

- 1) The nmea_navsat_driver is a popular ROS package that reads GPS data from the GPS module via serial communication (USB/serial port). It parses standard GPS messages (usually in **NMEA format**) and converts them into ROS messages.
sensor_msgs/NavSatFix – Provides latitude, longitude, altitude
- 2) ROS operates in Cartesian coordinate frames (x, y, z), but GPS gives geographic coordinates. navsat_transform_node (part of robot_localization package) is used.
- 3) Fuse GPS with other sensors use filter like Kalman filter.

